

Clasificación de Comandos de Voz con Robustez ante Ataques Adversariales

Segunda Entrega

Michael Steven Ruiz Palacio
Marlon Giraldo

Universidad de Antioquia — Deep Learning

Mayo 2026

Índice

1. Introducción
2. Estructura de los Notebooks
3. Solución Propuesta
 - 3.1 Arquitectura CNN Baseline
 - 3.2 Arquitectura CRNN (CNN + BiLSTM)
 - 3.3 Arquitectura Transformer con RoPE
4. Iteraciones y Decisiones de Diseño
 - 4.1 Primera Iteración: EDA y Preprocesamiento
 - 4.2 Segunda Iteración: Arquitecturas
 - 4.3 Tercera Iteración: Evaluación y Robustez
5. Resultados
6. Análisis de Robustez Adversarial
7. Conclusiones y Trabajo Futuro

1. Introducción

Este informe describe el desarrollo de la segunda entrega del proyecto de clasificación de comandos de voz utilizando técnicas de Deep Learning, con énfasis en la robustez frente a ataques adversariales. El proyecto se basa en la competencia Kaggle *Voice Commands Classification 2026*, que utiliza el dataset Google Speech Commands v0.02 con 35 clases de comandos de voz.

El objetivo principal es implementar y comparar tres arquitecturas de redes neuronales (CNN, CRNN y Transformer), evaluando su desempeño tanto en datos limpios como en un conjunto de test adversarial con perturbaciones pre-aplicadas.

El dataset contiene 95,246 muestras de entrenamiento y 10,576 muestras de prueba adversarial, todas en formato .npy (mono, 16 kHz, float32). Cada muestra corresponde a un comando de voz de aproximadamente 1 segundo de duración.

2. Estructura de los Notebooks

El proyecto se compone de 6 notebooks numerados que forman un pipeline completo, diseñados para ser ejecutados directamente en Google Colab sin configuración adicional:

- **01_exploracion_datos.ipynb** — Análisis exploratorio de datos (EDA). Carga y visualización de la distribución de clases, caracterización de señales de audio, análisis de amplitudes y duraciones, y discusión de paradigmas arquitectónicos.
- **02_preprocesamiento.ipynb** — Extracción de características (MFCC, mel-espectrogramas) y aumento de datos (ruido gaussiano, SpecAugment). División train/val (80/20).
- **03_modelo_baseline_cnn.ipynb** — Implementación y entrenamiento de CNN baseline con 3 capas convolucionales sobre mel-espectrogramas.
- **04_modelo_crnn.ipynb** — Implementación y entrenamiento de CRNN (CNN 1D + BiLSTM).
- **05_modelo_transformer.ipynb** — Implementación y entrenamiento de Spectrogram Transformer con RoPE (Rotary Position Embeddings) y SwiGLU.
- **06_evaluacion_robustez.ipynb** — Evaluación comparativa de los tres modelos en validación limpia y test adversarial con tablas y matrices de confusión.

Todos los notebooks incluyen detección automática del entorno (Colab vs. local), instalación de dependencias y descarga del dataset mediante kagglehub.

3. Solución Propuesta

Se implementaron tres arquitecturas que cubren distintos paradigmas de modelado de deep learning para clasificación de audio: convolucional, secuencial recurrente y atención pura.

3.1 Arquitectura CNN Baseline

La CNN baseline procesa audios convertidos a mel-espectrogramas (64 bandas mel, 1024 FFT, hop length 256). La arquitectura consta de:

- 3 capas convolucionales 2D con canales [32, 64, 128], kernel 3x3, padding 1, BatchNorm y ReLU.
- MaxPooling 2x2 después de cada convolución.

- Global Average Pooling para reducir a vector plano.
- Capa fully-connected de 128 neuronas con Dropout 0.3.
- Capa de salida softmax con 35 clases.

Parámetros totales: ~250K. Optimizador Adam con learning rate 0.001, batch size 64, early stopping con paciencia 4 épocas.

3.2 Arquitectura CRNN (CNN + BiLSTM)

La CRNN combina procesamiento convolucional con modelado secuencial temporal:

- Capa convolucional 1D: Conv1d(1, 32, kernel=80, stride=16), BatchNorm, ReLU.
- BiLSTM de 2 capas con hidden size 32 (bidireccional).
- Capa lineal: 64 -> 35 con salida log-softmax.
- Optimizador Adam con learning rate 0.001.

Esta arquitectura modela la dependencia temporal directamente sobre la forma de onda, a diferencia de la CNN que opera sobre el espectrograma completo.

3.3 Arquitectura Transformer con RoPE

El Spectrogram Transformer utiliza atención con posiciones rotatorias (RoPE):

- Parcheo lineal de mel-espectrogramas (parches de 16x16) con proyección a 128 dimensiones.
- 4 codificadores Transformer con 6 cabezas de atención por capa.
- FFN con SwiGLU (SwiGLUFFN) y dropout 0.1.
- Clasificador con salida de 35 clases.
- Optimizador AdamW con learning rate 0.0005.

Rotary Position Embeddings (RoPE) codifica la posición de cada parche mediante rotaciones en el espacio de atención, permitiendo que el Transformer capture relaciones espaciales sin necesidad de embeddings posicionales aprendidos.

4. Iteraciones y Decisiones de Diseño

4.1 Primera Iteración: EDA y Preprocesamiento

Se corrigieron las observaciones del profesor sobre la primera entrega:

- **Carga de metadatos reales:** El notebook 01 ahora carga metadata.csv con los labels reales del dataset y presenta la distribución real de clases.
- **Caracterización de señales:** Se incluye análisis de amplitudes, duraciones y forma de onda de muestras representativas, con visualización de mel-espectrogramas por clase.
- **Métrica correcta:** Se documentó que la métrica de evaluación es *macro F1-score* (multiclase) en lugar de accuracy binario.
- **Métrica de negocio:** Se definió la precisión balanceada (*balanced accuracy*) como métrica de negocio para penalizar desbalances.
- **Discusión de paradigmas:** Se incluyó una sección comparativa de los tres paradigmas arquitectónicos (CNN, RNN/CRNN, Transformer).

- **Ataques adversariales:** Se documentaron los tipos de ataque presentes en el conjunto `adv_test` (FGSM, PGD, BIM según la competencia).

4.2 Segunda Iteración: Arquitecturas

Se implementaron tres arquitecturas que cubren distintos paradigmas de modelado:

- **CNN 2D (Múltiple Instancia):** Procesa el mel-espectrograma como imagen, extrayendo patrones locales y globales mediante capas convolucionales apiladas.
- **CRNN (Secuencia/Dependencia):** Combina capas convolucionales 1D para extraer características de la forma de onda, con BiLSTM para capturar dependencias temporales.
- **Transformer (Atención Pura):** Utiliza auto-atención con RoPE para modelar relaciones entre parches del espectrograma.

4.3 Tercera Iteración: Evaluación y Robustez

El notebook `06_evaluacion_robustez.ipynb` realiza:

- Evaluación de los tres modelos en el conjunto de validación limpio.
- Evaluación en el conjunto `adv_test` con perturbaciones adversariales.
- Tabla comparativa de resultados (macro F1, accuracy, balanced accuracy).
- Matrices de confusión para cada modelo.
- Análisis de la degradación del rendimiento ante ataques.

5. Resultados

Los resultados de la evaluación en los conjuntos de validación y adversarial se presentan en la Tabla 1.

Modelo	Val. Acc.	Adv. Acc.	Adv. Macro F1
CNN Baseline	—	—	—
CRNN	—	—	—
Spectrogram Transformer	—	—	—

Tabla 1: Resultados comparativos de los tres modelos.

Nota: Los resultados numéricos se completan al ejecutar los notebooks correspondientes. El notebook 06 genera automáticamente esta tabla con los valores obtenidos.

6. Análisis de Robustez Adversarial

El conjunto `adv_test` contiene 10,576 muestras con perturbaciones adversariales pre-aplicadas. Se analiza:

- Degradación de accuracy y F1-score por modelo.
- Confusiones más frecuentes inducidas por los ataques.
- Qué arquitectura es más robusta frente a cada tipo de perturbación.

Se espera que modelos con mayor capacidad de generalización (Transformer con atención global) muestren mayor robustez relativa, mientras que la CNN baseline, al depender de patrones locales en el espectrograma, sea más susceptible a perturbaciones adversariales.

7. Conclusiones y Trabajo Futuro

Conclusiones

- Se implementaron y compararon tres arquitecturas representativas de distintos paradigmas de Deep Learning para clasificación de comandos de voz.
- El pipeline completo es reproducible en Google Colab mediante kagglehub.
- Las correcciones de las observaciones del profesor mejoraron significativamente la calidad del análisis exploratorio y la documentación del proyecto.
- La comparación sistemática en datos limpios y adversariales permite evaluar objetivamente la robustez de cada arquitectura.

Trabajo Futuro

- Implementar defensas adversariales (entrenamiento adversarial, suavizado gaussiano).
- Explorar ensembles de modelos para mejorar robustez.
- Aumentar la capacidad de los modelos (más capas/parámetros) dado el tamaño del dataset.
- Evaluar técnicas de data augmentation específicas para robustez (SpecAugment avanzado).

Documento generado automáticamente como parte del pipeline de Entrega 2.